AGENTS.md outperforms skills in our agent evals

> 📎 来源: *https://vercel.com/blog/agents-md-outperforms-skills-in-our-agent-evals*

7 min read

We expected skills to be the solution for teaching coding agents framework-specific knowledge. After building evals focused on Next.js 16 APIs, we found something unexpected.

A compressed 8KB docs index embedded directly in

```
AGENTS.md
```

achieved a 100% pass rate, while skills maxed out at 79% even with explicit instructions telling the agent to use them. Without those instructions, skills performed no better than having no documentation at all.

Here's what we tried, what we learned, and how you can set this up for your own Next.js projects.

AI coding agents rely on training data that becomes outdated. Next.js 16 introduces APIs like

```
'use cache'
```

,

```
connection()
```

, and

```
forbidden()
```

that aren't in current model training data. When agents don't know these APIs, they generate incorrect code or fall back to older patterns.

The reverse can also be true, where you're running an older Next.js version and the model suggests newer APIs that don't exist in your project yet. We wanted to fix this by giving agents access to version-matched documentation.

Before diving into results, a quick explanation of the two approaches we tested:

- **Skills** are an open standard for packaging domain knowledge that coding agents can use. A skill bundles prompts, tools, and documentation that an agent can invoke on demand. The idea is that the agent recognizes when it needs framework-specific help, invokes the skill, and gets access to relevant docs.

  `AGENTS.md`

  is a markdown file in your project root that provides persistent context to coding agents. Whatever you put in

  `AGENTS.md`

  is available to the agent on every turn, without the agent needing to decide to load it. Claude Code uses

  `CLAUDE.md`

  for the same purpose.

We built a Next.js docs skill and an

`AGENTS.md`

docs index, then ran them through our eval suite to see which performed better.

Skills seemed like the right abstraction. You package your framework docs into a skill, the agent invokes it when working on Next.js tasks, and you get correct code. Clean separation of concerns, minimal context overhead, and the agent only loads what it needs. There's even a growing directory of reusable skills at skills.sh.

We expected the agent to encounter a Next.js task, invoke the skill, read version-matched docs, and generate correct code.

Then we ran the evals.

In 56% of eval cases, the skill was never invoked. The agent had access to the documentation but didn't use it. Adding the skill produced no improvement over baseline:

CONFIGURATION	PASS RATE	VS BASELINE
Baseline (no docs)	53%	—
Skill (default behavior)	53%	+0pp

Zero improvement. The skill existed, the agent could use it, and the agent chose not to. On the detailed Build/Lint/Test breakdown, the skill actually performed worse than baseline on some metrics (58% vs 63% on tests), suggesting that an unused skill in the environment may introduce noise or distraction.

This isn't unique to our setup. Agents not reliably using available tools is a known limitation of current models.

We tried adding explicit instructions to

```
AGENTS.md
```

telling the agent to use the skill.

```
Before writing code, first explore the project structure,
then invoke the nextjs-doc skill for documentation.
```

Example instruction added to AGENTS.md to trigger skill usage.

This improved the trigger rate to 95%+ and boosted the pass rate to 79%.

CONFIGURATION	PASS RATE	VS BASELINE
Baseline (no docs)	53%	—
Skill (default behavior)	53%	+0pp
Skill with explicit instructions	79%	+26pp

A solid improvement. But we discovered something unexpected about how the instruction wording affected agent behavior.

Different wordings produced dramatically different results:

INSTRUCTION	BEHAVIOR	OUTCOME
"You MUST invoke the skill"	Reads docs first, anchors on doc patterns	Misses project context
"Explore project first, then invoke skill"	Builds mental model first, uses docs as reference	Better results

Same skill. Same docs. Different outcomes based on subtle wording changes.

In one eval (the

```
'use cache'
```

directive test), the "invoke first" approach wrote correct

```
page.tsx
```

but completely missed the required

```
next.config.ts
```

changes. The "explore first" approach got both.

This fragility concerned us. If small wording tweaks produce large behavioral swings, the approach feels brittle for production use.

Before drawing conclusions, we needed evals we could trust. Our initial test suite had ambiguous prompts, tests that validated implementation details rather than observable behavior, and a focus on APIs already in model training data. We weren't measuring what we actually cared about.

We hardened the eval suite by removing test leakage, resolving contradictions, and shifting to behavior-based assertions. Most importantly, we added tests targeting Next.js 16 APIs that aren't in model training data.

APIs in our focused eval suite:

```
connection()
```

for dynamic rendering

`'use cache'`

directive

`cacheLife()`

and

`cacheTag()`

`forbidden()`

and

`unauthorized()`

`proxy.ts`

for API proxying

- Async

  `cookies()`

  and

  `headers()`

  `after()`

  ,

  `updateTag()`

  ,

  `refresh()`

All the results that follow come from this hardened eval suite. Every configuration was judged against the same tests, with retries to rule out model variance.

What if we removed the decision entirely? Instead of hoping agents would invoke a skill, we could embed a docs index directly in

```
AGENTS.md
```

. Not the full documentation, just an index that tells the agent where to find specific doc files that match your project's Next.js version. The agent can then read those files as needed, getting version-accurate information whether you're on the latest release or maintaining an older project.

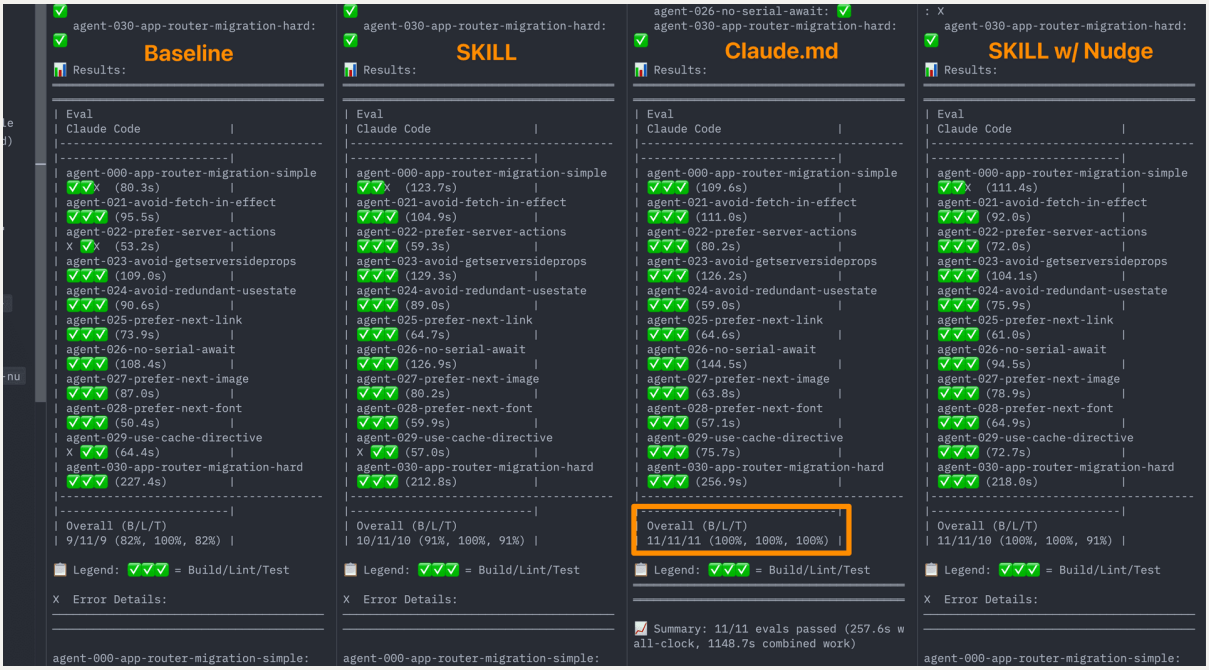
We added a key instruction to the injected content.

```
IMPORTANT: Prefer retrieval-led reasoning over pre-training-led
reasoning
for any Next.js tasks.
```

Key instruction embedded in the docs index

This tells the agent to consult the docs rather than rely on potentially outdated training data.

We ran the hardened eval suite across all four configurations:



Eval results across all four configurations. AGENTS.md (third column) achieved 100% across Build, Lint, and Test

Final pass rates:

CONFIGURATION	PASS RATE	VS BASELINE
Baseline (no docs)	53%	—
Skill (default behavior)	53%	+0pp
Skill with explicit instructions	79%	+26pp
`AGENTS.md` **docs index**	**100%**	**+47pp**

On the detailed breakdown,

> `AGENTS.md`

achieved perfect scores across Build, Lint, and Test.

CONFIGURATION	BUILD	LINT	TEST
Baseline	84%	95%	63%
Skill (default behavior)	84%	89%	58%
Skill with explicit instructions	95%	100%	84%
`AGENTS.md`	**100%**	**100%**	**100%**

This wasn't what we expected. The "dumb" approach (a static markdown file) outperformed the more sophisticated skill-based retrieval, even when we fine-tuned the skill triggers.

Why does passive context beat active retrieval?

Our working theory comes down to three factors.

1. **No decision point.** With

   > `AGENTS.md`

   , there's no moment where the agent must decide "should I look this up?" The information is already present.
2. **Consistent availability.** Skills load asynchronously and only when invoked.

`AGENTS.md`

content is in the system prompt for every turn.

3. **No ordering issues.** Skills create sequencing decisions (read docs first vs. explore project first). Passive context avoids this entirely.

Embedding docs in

`AGENTS.md`

risks bloating the context window. We addressed this with compression.

The initial docs injection was around 40KB. We compressed it down to 8KB (an 80% reduction) while maintaining the 100% pass rate. The compressed format uses a pipe-delimited structure that packs the docs index into minimal space:

```
[Next.js Docs Index]|root: ./.next-docs
|IMPORTANT: Prefer retrieval-led reasoning over pre-training-led reasoning
|01-app/01-getting-started:{01-installation.mdx,02-project-structure.mdx,...}
|01-app/02-building-your-application/01-routing:{01-defining-routes.mdx,...}
```

Minified docs in AGENTS.md

The full index covers every section of the Next.js documentation:

<!-- NEXT-AGENTS-MD-START -->[Next.js Docs Index]|root: ./.next-docs|IMPORTANT: Prefer retrieval-led reasoning over pre-training-led reasoning for any Next.js tasks.|If docs missing run: npx @judegao/next-agents-md|01-app/01-getting-started:{01-installation.mdx,02-project-structure.mdx,03-layouts-and-pages.mdx,04-linking-and-navigating.mdx,05-server-and-client-components.mdx,06-partial-prerendering.mdx,07-fetching-data.mdx,08-updating-data.mdx,09-caching-and-revalidating.mdx,10-error-handling.mdx,11-css.mdx,12-images.mdx,13-fonts.mdx,14-metadata-and-og-images.mdx,15-route-handlers-and-middleware.mdx,16-deploying.mdx,17-upgrading.mdx}|01-app/02-guides:{analytics.mdx,authentication.mdx,backend-for-frontend.mdx,caching.mdx,ci-build-caching.mdx,content-security-policy.mdx,css-in-js.mdx,custom-server.mdx,data-security.mdx,debugging.mdx,draft-mode.mdx,environment-variables.mdx,forms.mdx,incremental-static-regeneration.mdx,instrumentation.mdx,internationalization.mdx,json-ld.mdx,lazy-loading.mdx,local-development.mdx,mdx.mdx,memory-usage.mdx,multi-tenant.mdx,multi-zones.mdx,open-telemetry.mdx,package-bundling.mdx,prefetching.mdx,production-checklist.mdx,progressive-web-apps.mdx,redirecting.mdx,sass.mdx,scripts.mdx,self-hosting.mdx,single-page-applications.mdx,static-exports.mdx,tailwind-v3-css.mdx,third-party-libraries.mdx,videos.mdx}|01-app/02-guides/migrating:{app-router-migration.mdx,from-create-react-app.mdx,from-vite.mdx}|01-app/02-guides/testing:{cypress.mdx,jest.mdx,playwright.mdx,vitest.mdx}|01-app/02-guides/upgrading:{codemods.mdx,version-14.mdx,version-15.mdx}|01-app/03-api-reference:{07-edge.mdx,08-turbopack.mdx}|01-app/03-api-reference/01-directives:{use-cache.mdx,use-client.mdx,use-server.mdx}|01-app/03-api-reference/02-components:{font.mdx,form.mdx,image.mdx,link.mdx,script.mdx}|01-app/03-api-reference/03-file-conventions/01-metadata:{app-icons.mdx,manifest.mdx,opengraph-image.mdx,robots.mdx,sitemap.mdx}|01-app/03-api-reference/03-file-conventions:{default.mdx,dynamic-routes.mdx,error.mdx,forbidden.mdx,instrumentation-client.mdx,instrumentation.mdx,intercepting-routes.mdx,layout.mdx,loading.mdx,mdx-components.mdx,middleware.mdx,not-found.mdx,page.mdx,parallel-routes.mdx,public-folder.mdx,route-groups.mdx,route-segment-config.mdx,route.mdx,src-folder.mdx,template.mdx,unauthorized.mdx}|01-app/03-api-reference/04-functions:{after.mdx,cacheLife.mdx,cacheTag.mdx,connection.mdx,cookies.mdx,draft-mode.mdx,fetch.mdx,forbidden.mdx,generate-image-metadata.mdx,generate-metadata.mdx,generate-sitemaps.mdx,generate-static-params.mdx,generate-viewport.mdx,headers.mdx,image-response.mdx,next-request.mdx,next-response.mdx,not-found.mdx,permanentRedirect.mdx,redirect.mdx,revalidatePath.mdx,revalidateTag.mdx,unauthorized.mdx,unstable_cache.mdx,unstable_noStore.mdx,unstable_rethrow.mdx,use-link-status.mdx,use-params.mdx,use-pathname.mdx,use-report-web-vitals.mdx,use-router.mdx,use-search-params.mdx,use-selected-layout-segment.mdx,use-selected-layout-segments.mdx,userAgent.mdx}|01-app/03-api-reference/05-config/01-next-config-js:{allowedDevOrigins.mdx,appDir.mdx,assetPrefix.mdx,authInterrupts.mdx,basePath.mdx,browserDebugInfoInTerminal.mdx,cacheComponents.mdx,cacheLife.mdx,compress.mdx,crossOrigin.mdx,cssChunking.mdx,devIndicators.mdx,distDir.mdx,env.mdx,eslint.mdx,expireTime.mdx,exportPathMap.mdx,generateBuildId.mdx,generateEtags.mdx,headers.mdx,htmlLimitedBots.mdx,httpAgentOptions.mdx,images.mdx,incrementalCacheHandlerPath.mdx,inlineCss.mdx,logging.mdx,mdxRs.mdx,middlewareClientMaxBodySize.mdx,onDemandEntries.mdx,optimizePackageImports.mdx,output.mdx,pageExtensions.mdx,poweredByHeader.mdx,ppr.mdx,productionBrowserSourceMaps.mdx,reactCompiler.mdx,reactMaxHeadersLength.mdx,reactStrictMode.mdx,redirects.mdx,rewrites.mdx,sassOptions.mdx,serverActions.mdx,serverComponentsHmrCache.mdx,serverExternalPackages.mdx,staleTimes.mdx,staticGeneration.mdx,taint.mdx,trailingSlash.mdx,transpilePackages.mdx,turbopack.mdx,turbopackPersistentCaching.mdx,typedRoutes.mdx,typescript.mdx,urlImports.mdx,useCache.mdx,useLightningcss.mdx,viewTransition.mdx,webpack.mdx,webVitalsAttribution.mdx}|01-app/03-api-reference/05-config:{02-typescript.mdx,03-eslint.mdx}|01-app/03-api-reference/06-cli:{create-next-app.mdx,next.mdx}|02-pages/01-getting-started:{01-installation.mdx,02-project-structure.mdx,04-images.mdx,05-fonts.mdx,06-css.mdx,11-deploying.mdx}|02-pages/02-guides:{amp.mdx,analytics.mdx,authentication.mdx,babel.mdx,ci-build-caching.mdx,content-security-policy.mdx,css-in-js.mdx,custom-server.mdx,debugging.mdx,draft-mode.mdx,environment-variables.mdx,forms.mdx,incremental-static-regeneration.mdx,instrumentation.mdx,internationalization.mdx,lazy-loading.mdx,mdx.mdx,multi-zones.mdx,open-telemetry.mdx,package-bundling.mdx,post-css.mdx,preview-mode.mdx,production-checklist.mdx,redirecting.mdx,sass.mdx,scripts.mdx,self-hosting.mdx,static-exports.mdx,tailwind-v3-css.mdx,third-party-libraries.mdx}|02-pages/02-guides/migrating:{app-router-migration.mdx,from-create-react-app.mdx,from-vite.mdx}|02-pages/02-guides/testing:{cypress.mdx,jest.mdx,playwright.mdx,vitest.mdx}|02-pages/02-guides/upgrading:{codemods.mdx,version-10.mdx,version-11.mdx,version-12.mdx,version-13.mdx,version-14.mdx,version-9.mdx}|02-pages/03-building-your-application/01-routing:{01-pages-and-layouts.mdx,02-dynamic-routes.mdx,03-linking-and-navigating.mdx,05-custom-app.mdx,06-custom-document.mdx,07-api-routes.mdx,08-custom-error.mdx}|02-pages/03-building-your-application/02-rendering:{01-server-side-rendering.mdx,02-static-site-generation.mdx,04-automatic-static-optimization.mdx,05-client-side-rendering.mdx}|02-pages/03-building-your-application/03-data-fetching:{01-get-static-props.mdx,02-get-static-paths.mdx,03-forms-and-mutations.mdx,03-get-server-side-props.mdx,05-client-side.mdx}|02-pages/03-building-your-application/06-configuring:{12-error-handling.mdx}|02-pages/04-api-reference:{06-edge.mdx,08-turbopack.mdx}|02-pages/04-api-reference/01-components:{font.mdx,form.mdx,head.mdx,image-legacy.mdx,image.mdx,link.mdx,script.mdx}|02-pages/04-api-reference/02-file-conventions:{instrumentation.mdx,middleware.mdx,public-folder.mdx,src-folder.mdx}|02-pages/04-api-reference/03-functions:{get-initial-props.mdx,get-server-side-props.mdx,get-static-paths.mdx,get-static-props.mdx,next-request.mdx,next-response.mdx,use-amp.mdx,use-report-web-vitals.mdx,use-router.mdx,userAgent.mdx}|02-pages/04-api-reference/04-config/01-next-config-js:{allowedDevOrigins.mdx,assetPrefix.mdx,basePath.mdx,bundlePagesRouterDependencies.mdx,compress.mdx,crossOrigin.mdx,devIndicators.mdx,distDir.mdx,env.mdx,eslint.mdx,exportPathMap.mdx,generateBuildId.mdx,generateEtags.mdx,headers.mdx,httpAgentOptions.mdx,images.mdx,middlewareClientMaxBodySize.mdx,onDemandEntries.mdx,optimizePackageImports.mdx,output.mdx,pageExtensions.mdx,poweredByHeader.mdx,productionBrowserSourceMaps.mdx,reactStrictMode.mdx,redirects.mdx,rewrites.mdx,runtime-configuration.mdx,serverExternalPackages.mdx,trailingSlash.mdx,transpilePackages.mdx,turbo.mdx,typescript.mdx,urlImports.mdx,useLightningcss.mdx,webpack.mdx,webVitalsAttribution.mdx}|02-pages/04-api-reference/04-config:{01-typescript.mdx,02-eslint.mdx}|02-pages/04-api-reference/05-cli:{create-next-app.mdx,next.mdx}|03-architecture:{accessibility.mdx,fast-refresh.mdx,nextjs-compiler.mdx,supported-browsers.mdx}|04-community:{01-contribution-guide.mdx,02-rspack.mdx}<!-- NEXT-AGENTS-MD-END -->

The full compressed docs index. Each line maps a directory path to the doc files it contains

The agent knows where to find docs without having full content in context. When it needs specific information, it reads the relevant file from the

```
.next-docs/
```

directory.

One command sets this up for your Next.js project:

```
npx @next/codemod@canary agents-md
```

This functionality is part of the official```
@next/codemod

```
[package](https://github.com/vercel/next.js/pull/88961).

This command does three things:

1. Detects your Next.js version
2. Downloads matching documentation to

.next-docs/
```

3. Injects the compressed index into your

```
AGENTS.md
```

If you're using an agent that respects

```
AGENTS.md
```

(like Cursor or other tools), the same approach works.

Skills aren't useless. The

```
AGENTS.md
```

approach provides broad, horizontal improvements to how agents work with Next.js across all tasks. Skills work better for vertical, action-specific workflows that users explicitly trigger, like "upgrade my Next.js version," "migrate to the App Router," or applying framework best practices. The two approaches complement each other.

That said, for general framework knowledge, passive context currently outperforms on-demand retrieval. If you maintain a framework and want coding agents to generate correct code, consider providing an

```
AGENTS.md
```

snippet that users can add to their projects.

Practical recommendations:

- **Don't wait for skills to improve.** The gap may close as models get better at tool use, but results matter now.
- **Compress aggressively.** You don't need full docs in context. An index pointing to retrievable files works just as well.
- **Test with evals.** Build evals targeting APIs not in training data. That's where doc access matters most.
- **Design for retrieval.** Structure your docs so agents can find and read specific files rather than needing everything upfront.

The goal is to shift agents from pre-training-led reasoning to retrieval-led reasoning.

```
AGENTS.md
```

turns out to be the most reliable way to make that happen.

Research and evals by Jude Gao. CLI available at

```
npx @next/codemod@canary agents-md
```